

Cours 12 — Matrices : tableaux de tableaux

Semaine 12 ■ Manuel Hachette ch. 5 (§ 5.3, 5.4) ■ 2 × 1 h

Objectifs

- Représenter une grille par un tableau de tableaux
- Lire et écrire une case ; parcourir avec une double boucle
- Construire une matrice correctement (éviter le piège des alias)

1. Une grille = un tableau de lignes

Grille de jeu, image, tableau de données : beaucoup d'informations sont en **deux dimensions**. On les représente par un tableau dont chaque élément est une **ligne** :

```
m = [[5, 3, 8],
      [1, 4, 9]]      # 2 lignes, 3 colonnes
```

Accès : `m[i][j]` = ligne `i`, colonne `j` — **ligne d'abord**, et toujours depuis 0 :

```
m[0][2]      # 8
m[1][0] = 7   # modifie la case ligne 1, colonne 0
len(m)        # 2 : nombre de lignes
len(m[0])     # 3 : nombre de colonnes
```

2. Parcourir : la double boucle

```
for i in range(len(m)):      # chaque ligne
    for j in range(len(m[0])): # chaque colonne
        print(m[i][j])
```

L'ordre des boucles fixe l'ordre de visite : ici ligne par ligne. Pour visiter colonne par colonne, échanger les deux **for**.

Somme de toutes les cases, le motif accumulateur en 2D :

```
total = 0
for i in range(len(m)):
    for j in range(len(m[0])):
        total = total + m[i][j]
```

3. Construire une matrice : le piège

```
# FAUX : trois alias de LA MEME ligne
grille = [[0] * 3] * 3
grille[0][0] = 1      # modifie... les trois lignes !
```

C'est l'expérience troublante de la semaine 10 : `* 3` duplique la **référence** de la ligne, pas la ligne. Construction correcte, qui crée une nouvelle ligne à chaque tour :

```
grille = [[0] * 3 for _ in range(3)]
```

Règle simple : `[valeur] * n` est sûr pour des valeurs **immuables** (nombres, chaînes); dès que les éléments sont des **listes**, utiliser une compréhension.

4. Deux exemples à connaître

Recherche du maximum d'une matrice : même logique qu'en 1D, avec la double boucle.

Transposée (lignes $\leftrightarrow$ colonnes), utile pour « retourner » des données :

```
def transposee(m):  
    """Renvoie la transposée de m (lignes <-> colonnes)."""  
    n_lignes = len(m)  
    n_cols = len(m[0])  
    t = [[0] * n_lignes for _ in range(n_cols)]  
    for i in range(n_lignes):  
        for j in range(n_cols):  
            t[j][i] = m[i][j]  
    return t
```

À retenir

- `m[i][j]` : ligne `i`, colonne `j`; dimensions : `len(m)` et `len(m[0])`.
- Parcours complet = double boucle; l'ordre des `for` fixe l'ordre de visite.
- Construire : `[[0] * p for _ in range(n)]` — jamais `[[0] * p] * n`.
- Une image en niveaux de gris est une matrice d'entiers 0-255 (TP du jour).